

深度学习大作业实验报告

一、实验背景

这次大作业要求我们用华为的 openPangu-Embedded-7B 模型在 HotpotQA 数据集上实现四种不同的方法：Prompt、SFT、RAG 和 Agent，最后用 EM 和 F1 对比效果。

HotpotQA 是个多跳问答数据集，一个问题通常要查好几篇文档才能回答，不是那种一问一答的简单问题。比如问“梵德格拉夫发电机的设计者在哪个学校教书”，先得知道谁设计了发电机，再查这人在哪教书。

用的是 fullwiki 设置，模型不能直接用数据里自带的上下文，得自己想办法检索信息，这样更贴近真实场景。测试集一共 200 条数据，其中 160 条 bridge 类型、40 条 comparison 类型。

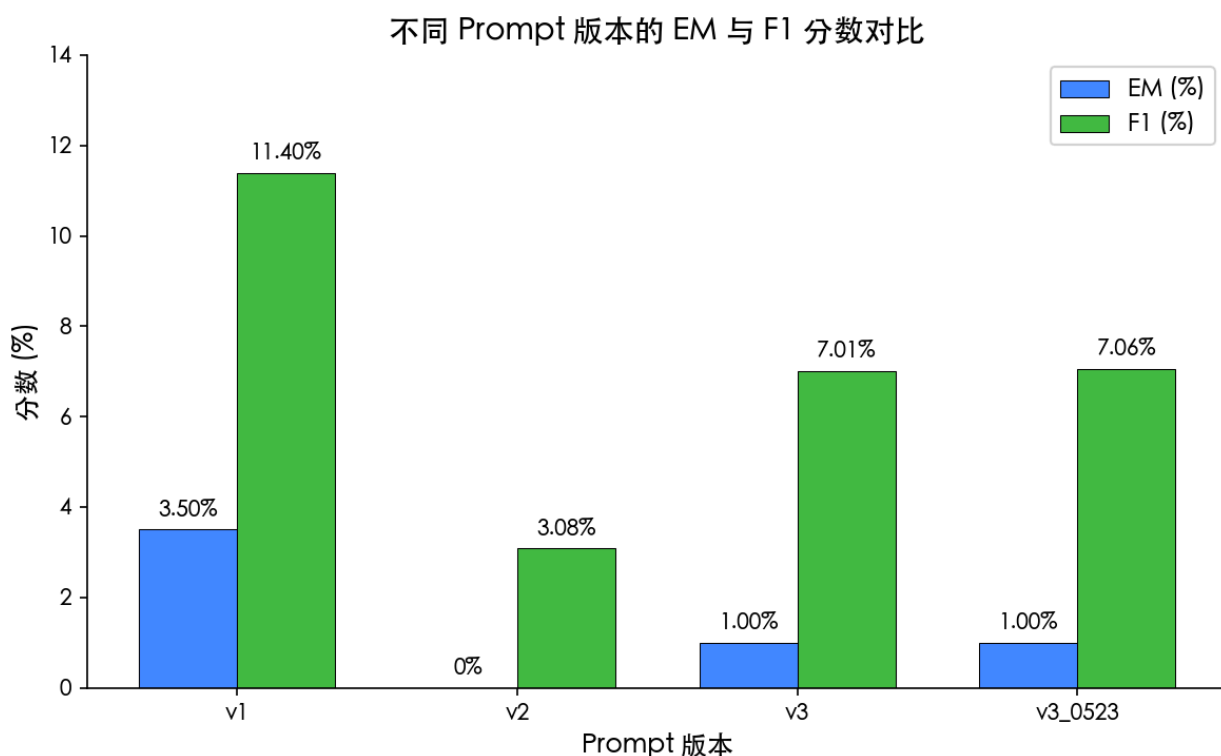
二、Prompt

这个最简单，就是直接给 prompt 让模型回答问题，不检索不微调，纯粹看模型自己知道多少。跑这个主要是为了给后面几种方法做对比基准。

实验过程：先在华为云上启动了 vLLM 推理服务，加载 openPangu-7B 模型，vLLM 要做一些初始化工作。加载完成后就开始给每条数据构造 prompt 然后批量推理。

一开始直接提问发现不行，模型默认输出 thinking 模式的 <think> 标签，得在 prompt 末尾加 /no_think 关掉。还发现它经常输出 [unused16][unused17] 这类特殊 token，得做额外清洗。我试了不同的提取策略，最后是先清理特殊 token，然后尝试匹配 <answer> 标签，匹配不上就取最后一行的第一个非空行。

生成参数试了好几版：temperature 设 0，repetition_penalty 从无到 1.05，max_tokens 从 128 调到 256。



Prompt 各版本效果对比

实验结果：v1 (tp=4, 无 repetition_penalty) EM 3.50% F1 11.40%，虽然指标最高但其实是有问题的。v1 没加重重复惩罚，模型会一直生成不停，输出内容很长。F1 是按 token 匹配算的，输出越长蒙对词的概率就越高，所以 F1 被虚假拉高了。但 EM 是精确匹配，跟长度无关，为什么 v1 的 EM 也最高呢？其实 3.5% 和 1% 差的也就是 7 条对 2 条的区别，200 条数据里多蒙对 5 条很可能是随机波动。

v2 (tp=1, max_tokens=128) 结果最差 EM 0% F1 3.08%，max_tokens 设太短了输出直接被截断。v3 加了惩罚后正常了一些，v3_0523 是最稳定的一版，EM 1%，F1 7.06%。推理速度挺快的（200 条 7.9 秒，25 条/秒）。bridge 类型的 EM 0.62% F1 5.31%，comparison 类型的 EM 2.50% F1 14.08%，comparison 稍微好一点可能是因为 yes/no 类的问题模型猜对的概率大一些。

模型输出的质量不太行，很多答案都是胡说八道。比如第一个样本问"five-and-dime 商店历史建筑在哪个城市"，模型答出个 Safeway 超市在 San Jose，实际上正确答案是 Watertown, New York，完全不沾边。还有问"梵德格拉夫发电机的设计者在哪儿教书"，模型编了个 Alexander F. Graham 出来说在 UC Berkeley，实际是 Robert J. Van de Graaff 在普林斯顿和 MIT。

三、SFT

SFT 就是用 HotpotQA 训练集对模型做 LoRA 微调，让模型学着回答多跳问题。

实验过程：SFT 的实验流程比 Prompt 复杂得多，主要有这几步：

第一步是数据预处理。用 convert_hotpotqa_to_sft.py 把 HotpotQA 的训练数据转成 MindSpeed-LLM 需要的 packed 格式。这一步跑完之后会生成 input_ids、labels、attention_mask 等文件。

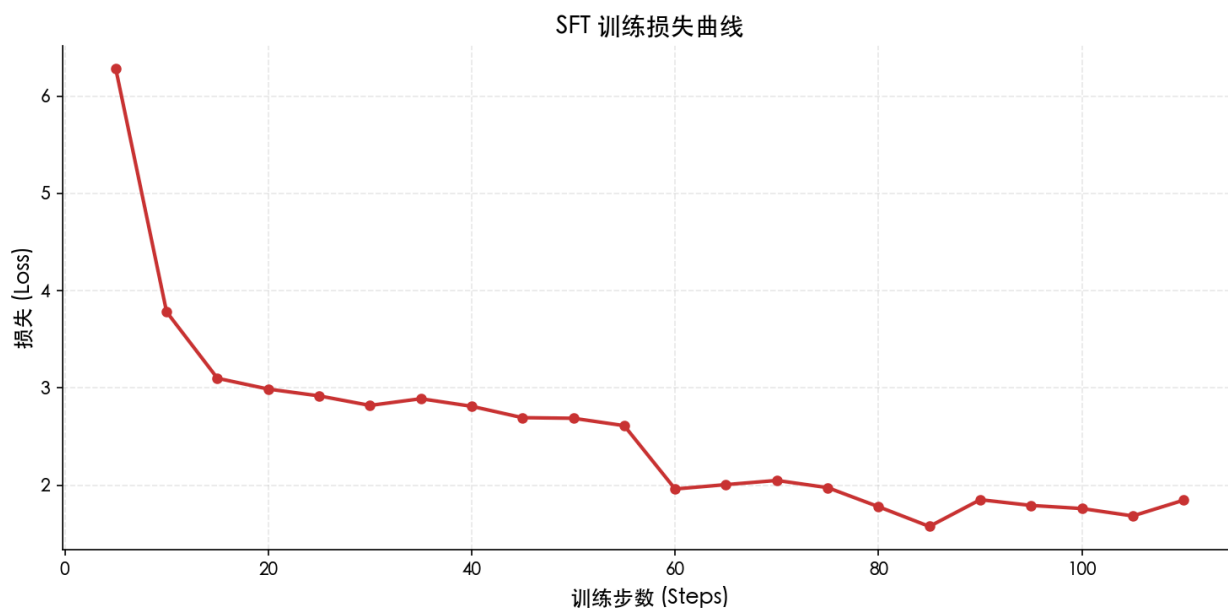
第二步是模型格式转换。openPangu 原始的是 huggingface 格式，但 MindSpeed-LLM 训练需要 mcore 格式（这个是华为昇腾适配的格式，参数会根据 Tensor Parallel 自动切分）。用 convert_ckpt.py

来转，`--target-tensor-parallel-size` 设成 4（因为我们用 4 张 NPU）。

第三步是训练。用 `tune_pangu_7b_full_ptd.sh` 脚本启动训练。训练数据是从 HotpotQA 训练集（总共 90,564 条）里取了一部分，实际用了 437 条原始数据，经过 packing 后变成 880 条训练样本（每条 8192 token），对应训练 110 步、`global_batch_size=8`，数据量不到完整训练集的 0.5%。全部用来训练没做验证集划分。LoRA 参数 `r=16`，`alpha=32`，学习率 `1e-4`，cosine 衰减到 `2e-6`，`weight_decay 0.1`。`batch_size=8`（`micro_batch=1`，`gradient_accumulation=8`），bf16 精度，用了 `flash_attn`。4 张 NPU 并行（`tp=4`），总共大概 15 分钟。

第四步训练完了之后还要把 mcore 格式转回 hf 格式才能做推理。

第五步推理评测，用转换后的模型在 200 条测试集上跑，计算 EM 和 F1。



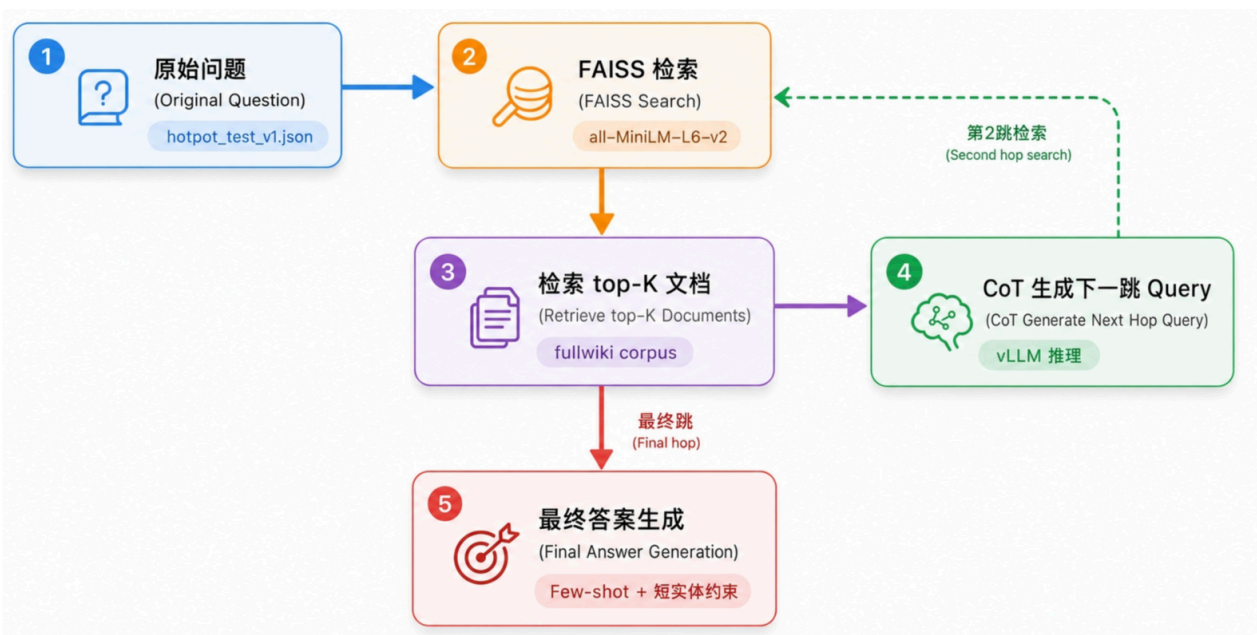
SFT 训练 Loss 收敛曲线

实验结果：训练本身是正常的，loss 从 6.28 降到 1.85，`grad_norm` 也稳定（1.4~48.8 之间），没有出现 NaN。但推理结果让人失望，EM 0%，F1 0.37%，比纯 Prompt 还低。我猜有几个原因：一是 LoRA rank 16 可能不够，7B 模型这么低的 rank 学不到多跳推理的复杂模式；二是用了 437 条原始数据（pack 成 880 条训练样本），不到完整 HotpotQA 训练集 90,564 条的 0.5%，数据量确实太少了；三是推理时 prompt 格式跟训练数据格式没对齐好。另外 mcore 转 hf 的过程中可能有精度损失也不准。

四、RAG

RAG 的做法是加一个检索模块，让模型先查资料再回答。

实验过程：首先需要构建向量检索库。我们从 HotpotQA 官方提供的 fullwiki 语料（几十万篇 wikipedia 段落）用 `all-MiniLM-L6-v2` 做 embedding，然后用 FAISS 的 `IndexFlatIP` 建索引，存成本地文件 `faiss_fullwiki_index.bin`（这样下次就不用重新建了）。



RAG 多跳问答流水线

检索流水线是分多跳的。第一跳用原始问题去 FAISS 里检索 top-K 篇文档，然后让 openPangu 模型基于检索结果生成 CoT，想想还需要什么信息才能回答。CoT 出来后拼接到原始问题上作为下一跳的 Query，再检索一次。最后一跳用 few-shot 格式让模型输出短实体答案。中间检索的 max_tokens 设 100，repetition_penalty 1.15。最后答案生成的 max_tokens 设 30，碰到换行就停。答案提取用了首行优先策略——如果第一行小于等于 6 个词就直接用，否则从所有行里找引导词匹配。

实验结果：EM 19.50%，F1 27.40%，比 Prompt 好太多了，说明引入检索确实有用。200 条里有 16 条完全没输出答案（prediction 是空字符串），而且检索质量不太稳定。有些问题的检索结果完全跑偏了，比如 five-and-dime 问题检索出来的都是摩天大楼不是商店，这跟问题语义理解不到位有关，embedding 模型的选型很关键。

在实验的过程中也遇到了很多问题，记录如下：

1. 模型输出过长导致答案被淹没：openPangu 在 Answer: 之后往往先输出正确的短答案，然后接着输出一大段解释（Wait, but... / Therefore...），导致 EM 极低。解决方案是采用首行优先提取策略——优先取第一个小于等于 6 词的非废话行作为答案，配合 max_tokens=30 和 stop 换行符强制截断。
2. 模型生成退化循环（Attention Collapse）：部分样本出现 individual individual individual 的无限重复输出。通过将 repetition_penalty 从 1.05 提升到 1.15 彻底解决。
3. 检索召回质量瓶颈：all-MiniLM-L6-v2（22M 参数）语义召回能力有限，大量问题的正确文档根本不在 Top-K 结果中（如问 F.W. Woolworth 却召回摩天大楼文档）。这是整个 RAG 系统效果上限的问题。

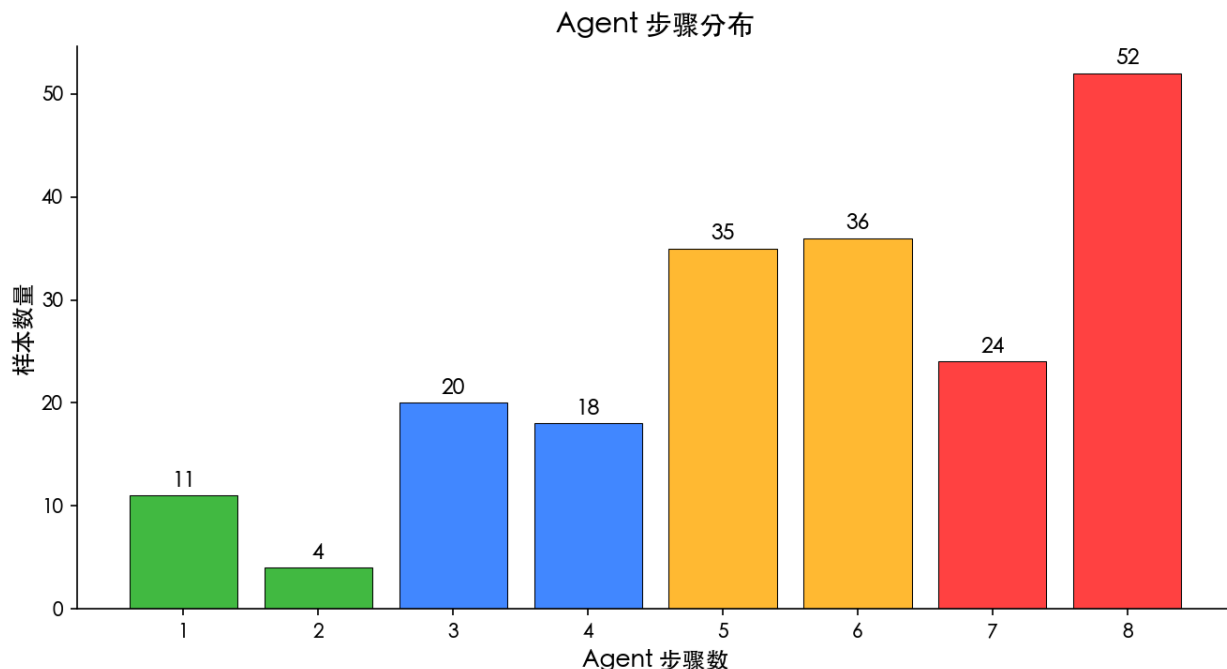
五、Agent

Agent 用的是 ReAct 范式，让模型自己决定下一步怎么做——搜索、再搜索、还是给出答案。

实验过程：先启动 vLLM 推理服务（TP=4），加载 openPangu-7B。然后配置 Serper API 的 key（用于 WebSearch）。Agent 的运行逻辑是：每个 step 把所有还没完成的样本打包成 batch 提交给 vLLM，解析

返回结果。如果返回的 action 是 Search，就用多线程并发去 FAISS 检索；如果是 WebSearch，就调 Serper API 搜 Google；如果是 Finish，就记录答案并标记样本完成。

我们还设计了一个预热策略——第一步无条件给所有样本先搜一次本地知识库，把结果预填进 scratchpad，这样模型一开始就有东西可以思考，不会直接格式错误。实验证明这个预热策略很关键，不加的话第一步格式错误率会很高。最大步数设了 8 步，超过就强制 Few-shot 提取答案。



Agent 步数分布

实验结果：EM 21.50%，F1 29.82%，四种里最高。步数分布来看：1 步 11 条（只做了预热检索来不及及推理，这些都是空预测的），3–5 步 73 条，6–8 步 112 条，有 52 条跑满了 8 步才强制结束。说明大部分问题模型需要多步才能搞定，但也有不少是到了步数上限还没找到正确答案。

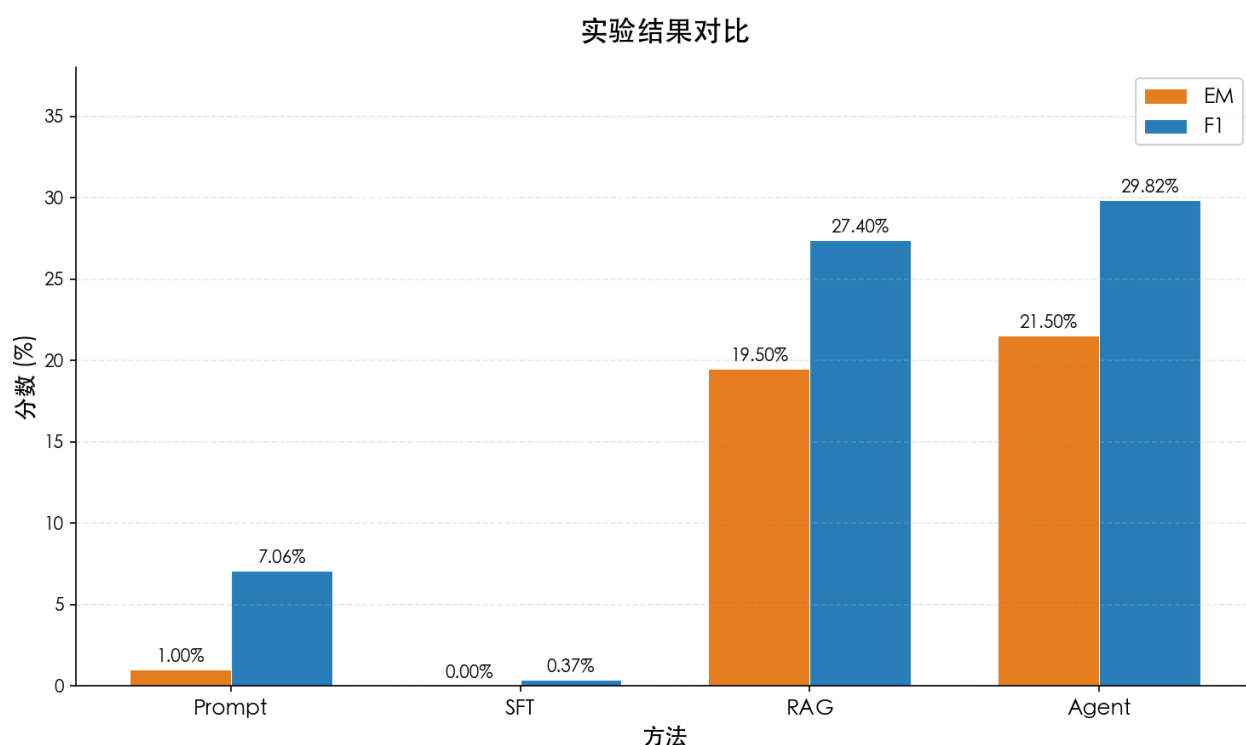
预热策略确实有效，但不预热的话第一步很多样本会输出不合格式的内容。但也有副作用——第一跳检索结果如果不相关，后面会被带偏。Serper API 的 WebSearch 挺有用，不过有调用次数限制。

实验过程中遇到的问题记录如下：

1. Prompt 超长导致 vLLM 报错：Agent 迭代多步后，scratchpad（累积的 Thought + Action + Observation 历史）越来越长，最终导致 prompt 总长度超过 `max_model_len=4096`，报出 `ValueError: decoder prompt length 4149 > maximum model length 4096`。解决方案：将 `max_model_len` 提升到 8192（openPangu 支持 8K 上下文）。
2. `max_steps=1` 时所有预测为空：Agent 第一步选择了 Search 而非 Finish，循环结束后没有机会再生成答案。解决方案：增加 Forced Final Answer Extraction 部分——循环结束后对未完成样本，用 Few-shot 格式从已有的 observation 中强制提取答案。
3. 模型 60% 的样本第一步就 `FormatError`：openPangu-7B 对 ReAct 格式的遵循能力弱，经常不按 `Thought: ... Action: Search[...]` 格式输出。解决方案：增加 Pre-fetch 预热阶段，在 ReAct 循环开始前无条件对所有样本执行一次批量检索并将结果预填入 scratchpad，这样模型从第一步开始就已有上下文可以推理。同时在 `FormatError` 时从 `raw_output` 中匹配 `the answer is X` 等模式直接提取答案。

4. WebSearch 从未被调用（即使配置了 Serper Key）：openPangu 不认识 WebSearch 这个工具名，模型永远只输出 Search[...]。解决方案：实现 Auto-Escalation 自动升级策略——Step 1 的 Search 走本地 FAISS，Step 2+ 自动升级为 Serper WebSearch；FormatError 兜底也直接走 WebSearch。
5. Serper API 429 限流（77/200 次超时）：原来的并发线程数为 16，免费 Key 承受不住。解决方案：降低 max_workers 到 4，每次请求前加入 300ms 延迟，遇到 429 时等待 2 秒后自动重试。
6. 多步迭代后效果反而变差：本地 FAISS 召回率低，多次错误检索会把 Fallback 引向更错误的方向（如最后一次 observation 是无关文档，Fallback 就从中提取了错误答案）。解决方案：Fallback 改为从所有历史 observation 中选择内容最丰富且无错误的那条；同时实验发现 max_steps=2 是最优策略（Step 1 本地检索 + Step 2 WebSearch 补充）。

六、对比分析



实验结果对比 (F1 Score)

从结果看，最关键的发现就是：**不检索根本没法做**。引入了检索的 RAG 和 Agent 效果（F1 27–30%）比纯靠模型自己的 Prompt（7%）好太多了，差了好几倍。这说明 openPangu-7B 的内部知识对多跳问答确实不够用，7B 参数的模型在需要整合多篇文档信息的任务上有天然的能力边界。

SFT 这次是最让人失望的，花了不少时间配置训练环境、数据转换、格式转换，结果反而更差了。我觉得核心问题是数据量和 LoRA 的配置。如果有机会重做的话，我会试试用全部 9 万条训练数据，LoRA rank 调大一点（比如 64 或 128），或者直接用全参数微调。另外训练数据的格式也需要跟推理时的 prompt 格式严格对齐。

RAG 和 Agent 的差距没有想象中大（F1 差 2% 左右）。说明在这个任务上，检索质量比决策机制更关键——不管你用 RAG 的两跳还是 Agent 的多步迭代，如果检索不到正确的信息，后面的推理都是白搭。Agent 的优势在于更灵活，可以自己决定搜索方向，但这也意味着它有可能在错误的方向上浪费很多步数。

四种方法的优缺点总结：

Prompt 最简单，零成本，但效果最差，基本只适合当 baseline。SFT 成本最高（要训练、要格式转换），效果还不稳定，小数据量下不建议尝试。RAG 性价比最高，不需要训练，搭一套检索系统后效果提升明显。Agent 效果最好但实现复杂，而且需要外部 API（Serper），稳定性受限于工具的可用性。

七、总结

这次实验让我对四种 LLM 使用方式有了比较直观的认识，也踩了一些坑：

1. 多跳问答这个任务对 7B 模型来说还是太难了，没有外部知识基本做不了。检索是必须的，加入后 F1 从 7% 提高到了 30%。
2. LoRA SFT 在小数据量下效果不好，甚至起反效果。感觉 SFT 更适合数据量大、任务格式固定的场景，而不是这种需要推理能力的任务。
3. Agent (F1 29.82%) 和 RAG (F1 27.40%) 效果接近，在 7B 这个规模上 Agent 的规划能力有限，迭代搜索有时候还不如直接固定检索两跳靠谱。
4. 检索质量是整个系统的瓶颈。不管哪种方法，检索结果不对后面就全错了。embedding 模型的选择、检索策略的设计都很关键。

如果后续还能继续做的话，我会尝试：用更大的 SFT 数据量（全量 9 万条）、用更好的 embedding 模型（bge-m3 之类的）、给 Agent 加更多工具（比如 Wikipedia API、结构化知识库查询），效果应该还能再往上走。